

Linux Privilege Escalation: Capabilities

Your SUID guide closed by calling capabilities "SUID's more granular sibling" — this guide is that promised contrast. SUID is all-or-nothing: a binary either runs entirely as root, or it doesn't run with elevated privileges at all. Capabilities split root's traditionally monolithic power into dozens of **individually grantable permissions**, letting a binary get exactly the narrow slice of root power it actually needs — which sounds like a strict security improvement, and largely is, but creates its own distinct exploitation surface once you understand which individual capabilities are dangerous on their own. Worth locking in the distinction immediately: **Part 1 covers what capabilities actually are and how to enumerate them — Part 2 covers the specific dangerous capabilities and how each is individually exploited**, since (unlike SUID's single binary bit) each capability needs its own distinct exploitation technique.

1. Why Capabilities Exist — The Foundation

Traditionally, Unix permission checks boiled down to a single binary question: is this process running as **root** (UID 0), or not? Root could do literally everything; every other UID was subject to full permission checking. This binary model is exactly what makes SUID (your previous guide) so dangerous — there's no middle ground between "no special power" and "all of root's power."

Traditional model: root (UID 0) → CAN do everything any other UID → SUBJECT to normal permission checks

Capabilities model: root's traditional power is split into ~40 DISTINCT, individually-grantable units (CAP_NET_BIND_SERVICE, CAP_SYS_ADMIN, CAP_DAC_OVERRIDE, CAP_SETUID, and so on)

The single fact this guide is built around — a non-root process can be granted ONE specific capability (say, the ability to bind to a low network port) without being granted root's full power, which is exactly the security improvement capabilities were designed to deliver — but several individual capabilities are, on their own, functionally equivalent to full root access once

you understand what they actually permit, meaning the "narrow slice" isn't always narrow at all.

PART 1: Understanding & Enumerating Capabilities

2. How Capabilities Attach to a Process — Two Places to Check

Capabilities can be assigned directly to a **file** (persisting across every execution of that binary, similar in spirit to SUID) or held by a **running process** dynamically:

```
# File-based capabilities (persistent, attached to the binary itself)
getcap -r / 2>/dev/null

# Process-based capabilities (the CURRENT shell/process's capability set)
cat /proc/self/status | grep Cap
capsh --print
```

```
getcap -r / output example:
/usr/bin/ping = cap_net_raw+ep
/usr/bin/python3.9 = cap_setuid+ep      <- immediately suspicious,
                                         covered in Section 5

+ep suffix meaning:
e = Effective (currently active/usable)
p = Permitted (allowed to be made effective)
i = Inheritable (passed down to child processes, less common to see set)
```

3. Capability Sets — Why the Suffix Letters Matter

A capability can be present in a process's capability set without being ACTIVELY USABLE at that moment - the distinction between "permitted" and "effective" matters for exploitation:

Permitted only (p, no e) → the process COULD use this capability if it explicitly raises it, but isn't currently exercising it

Effective (e) → the capability is ACTIVELY in force right now for this process

Most exploitation techniques in Part 2 require the capability to be

BOTH permitted and effective (or trivially raisable to effective)
on the target binary - always confirm the +ep suffix, not just the
capability name alone, before assuming a finding is exploitable

4. Distinguishing Expected vs Suspicious Capabilities

Similar to your SUID guide's Section 3 distinction — most systems have a small, expected set of capability assignments (`ping` needing `cap_net_raw`, for instance, to construct raw ICMP packets without full root). The goal is spotting capabilities on binaries that shouldn't plausibly need them — especially interpreters (`python`, `perl`, `ruby`, `node`) or general-purpose tools, which is almost always a misconfiguration rather than an intentional, narrowly-scoped grant.

PART 2: Exploiting Specific Dangerous Capabilities

5. CAP_SETUID — Direct Path to Root

Grants the ability to change a process's UID arbitrarily — if held by an interpreter or any binary capable of executing arbitrary code, this is functionally identical to SUID-root, since the process can simply set its own UID to 0:

```
# if python3 has cap_setuid+ep
python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'
```

This is the single most immediately dangerous capability to find on a general-purpose interpreter - it collapses the entire "granular permission" premise of capabilities back into full root, in one line

6. CAP_SETGID — Group-Level Equivalent

Same principle as Section 5, one level down — grants the ability to change a process's GID, useful directly when a target file/resource is protected by group ownership rather than requiring full root, and frequently chained alongside CAP_SETUID when both are present.

7. CAP_DAC_OVERRIDE — Bypassing File Permission Checks

Grants the ability to bypass discretionary access control (DAC) — i.e., ordinary file read/write/execute permission checks — entirely, regardless of the file's

actual permission bits:

```
# if a binary with cap_dac_override+ep can be leveraged to
# read/write
# arbitrary files, e.g. via python
python3 -c 'open("/etc/shadow", "r").read()'
python3 -c 'open("/etc/passwd", "a").write("backdoor::0:
0::/root:/bin/bash\n")'
```

Directly enables the SAME /etc/passwd overwrite technique your Writable Files guide covers in more depth, here reached via a capability instead of an actual filesystem permission misconfiguration

8. CAP_SYS_ADMIN — The "Almost Everything" Capability

Widely considered the single most dangerous capability to encounter, since it's a genuine catch-all covering a large number of administrative operations that were never cleanly separable into their own individual capabilities — mounting filesystems, and critically, several container-escape-relevant operations:

```
# mount-based exploitation if cap_sys_admin+ep is present
mkdir /tmp/cgrp && mount -t cgroup -o memory cgroup /tmp/cg
rp
# (specific technique varies significantly by exact system/
kernel
# version and available cgroup/mount configuration - treat
this as
# a strong signal to research the CURRENT specific techniq
ue for
# the target kernel version rather than a single fixed rec
ipe)
```

Frequently the deciding factor in container-escape scenarios specifically, since a containerized process retaining CAP_SYS_ADMIN can often manipulate the container's own mount namespace in ways that reach the underlying host.

9. CAP_NET_ADMIN — Network-Level Escalation Paths

Grants broad network configuration control (interface configuration, routing table manipulation, firewall rule changes) — less directly a "get root" capability on its own compared to Sections 5-8, but valuable for network-level manipulation (traffic interception via interface reconfiguration, disabling firewall rules protecting other services) that can be chained into a broader escalation path.

10. CAP_CHOWN and CAP_FOWNER — Ownership Manipulation

`CAP_CHOWN` allows changing file ownership arbitrarily; `CAP_FOWNER` bypasses ownership-based permission checks similarly to Section 7's `DAC_OVERRIDE` but specifically for ownership-dependent operations (like removing the sticky bit on `/tmp` or modifying files you don't own). Both are commonly chained with Section 5/6's `SETUID/SETGID` findings to take full ownership of a target resource before manipulating it.

11. Testing Methodology

1. Enumerate file-based capabilities system-wide (Section 2's `getcap -r /`) - this is the primary discovery step, analogous to your SUID guide's `find -perm -4000`
2. For each result, confirm the `+ep` suffix is present (Section 3) - a permitted-only capability without effective status needs additional work to actually leverage
3. Cross-reference the specific capability name against Sections 5-10 to determine what it actually enables - don't assume based on the name alone, verify against current documentation, since capability semantics are precise and easy to misjudge
4. For interpreters/general-purpose tools specifically (Section 4), check `CAP_SETUID` and `CAP_DAC_OVERRIDE` first - these are the fastest, most reliable paths to a root shell when present
5. `GTF0Bins` (referenced in your SUID guide) also documents capability-specific exploitation techniques for many standard binaries - check there before assuming manual exploitation is required

12. Prevention

- Audit file capabilities regularly (`getcap -r /`) with the same discipline as SUID auditing - remove capabilities from binaries that don't demonstrably need them
- NEVER grant `CAP_SETUID`, `CAP_SETGID`, or `CAP_DAC_OVERRIDE` to a general-purpose interpreter (python, perl, node, etc.) - these effectively grant full root regardless of the

"granular" intention behind capabilities as a mechanism

- Treat CAP_SYS_ADMIN as equivalent to full root when assessing risk - it is not meaningfully more restrictive in practice despite being "just one capability" among many
- Prefer the MOST SPECIFIC capability that satisfies the actual requirement (e.g. CAP_NET_BIND_SERVICE for a service binding to a low port, rather than reaching for CAP_SYS_ADMIN or a broader grant "to be safe")
- In containerized environments specifically, explicitly DROP all capabilities by default and add back only the minimum required set per container/workload, rather than inheriting a permissive default capability set

13. Practical Lab Example

HTB Academy's Linux Privilege Escalation module has a dedicated capabilities section; TryHackMe's privilege escalation rooms frequently include a capability-based box specifically distinct from their SUID-focused ones.

Suggested practice order:

1. On a lab VM, run `getcap -r /` and note the baseline expected set
2. Deliberately assign `cap_setuid+ep` to `/usr/bin/python3` yourself (`setcap cap_setuid+ep /usr/bin/python3`) and practice Section 5's one-liner
3. Repeat with `cap_dac_override+ep` and practice reading `/etc/shadow` directly (Section 7), then compare both techniques' simplicity against your SUID guide's more involved GTF0Bins/LD_PRELOAD techniques for the equivalent outcome

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-269 (Improper Privilege Management - same root CWE as your SUID guide, since capabilities are a different mechanism for the identical underlying misconfiguration category)
OWASP Top 10:2025	A02:2025 Security Misconfiguration
CVSS	High/Critical for CAP_SETUID/CAP_DAC_OVERRIDE/CAP_SYS_ADMIN findings specifically (functionally equivalent to root), lower for narrower capabilities like CAP_NET_BIND_SERVICE alone
Cyber Kill Chain	Privilege Escalation

Framework	Mapping
MITRE ATT&CK	T1548 (Abuse Elevation Control Mechanism) - capabilities sit under the same broader technique family as SUID (T1548.001) though without their own dedicated sub-technique ID as of current ATT&CK versions

Quick Reference Cheat Sheet

ENUMERATE:

```
getcap -r / 2>/dev/null      → file-based capabilities, system-wide
cat /proc/self/status | grep Cap → current process's capabilities
capsh --print                → human-readable current capability decode
```

DANGEROUS CAPABILITIES (roughly most → least immediately critical):

```
cap_setuid      → set own UID to 0 directly (near-instant root)
cap_dac_override → bypass ALL file permission checks
cap_sys_admin   → broad admin catch-all, container-escape relevant
cap_setgid      → group-level equivalent of setuid
cap_fowner      → bypass ownership-based permission checks
cap_chown       → arbitrary ownership changes
cap_net_admin   → network reconfiguration, less directly
                  "root" but a strong chaining primitive
```

ONE-LINERS (given cap_setuid+ep on python3):

```
python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'
```

Root lesson: capabilities are a genuine security improvement over SUID's all-or-nothing model in principle, but SEVERAL individual capabilities collapse that granularity right back to "equivalent to full root" - the mechanism changed, but the auditing discipline (check for anything beyond the minimum necessary grant) is identical to SUID's.